# Trust Boundaries and the Limits of Pattern-Based Prompt Injection Detection

Anonymous Author(s)

## Abstract

We evaluate a two-tier prompt injection guardrail, an Aho–Corasick blocklist followed by a fine-tuned DeBERTa-v3 classifier, and report a sequence of measurements that each revise its apparent performance downward. On the 271-sample corpus its rules were tuned against, the system reaches 1.000 recall at zero false positives. Held out from that same corpus it reaches 0.789. On Open-Prompt-Injection it reaches 0.40, and on BIPIA 0.48. The shortfall is not a tuning deficit. Attacks that announce themselves with override vocabulary are caught almost perfectly, and attacks that inject a well-formed instruction carrying no override phrasing are largely invisible. We then show that one bit of deployment context, whether text arrived as a user turn or as data the application is processing, moves recall on Open-Prompt-Injection from 0.40 to 1.00 at a 0.6% false positive rate, because the same sentence is a legitimate request on one side of that boundary and an attack on the other. Two further results have practical consequences for anyone deploying such a system. INT8 dynamic quantization silently reduced the classifier tier to a constant SAFE prediction, which a fail-open policy rendered invisible. And on NotInject the deterministic tier blocks 0 of 339 benign trigger-word prompts while the learned tier blocks 137, which makes the learned tier unshippable as a blocking stage. Artifacts and every measurement script are public.

## CCS Concepts

• **Security and privacy** → **Network security**; • **Computing methodologies** → *Machine learning*.

## Keywords

prompt injection; LLM security; guardrails; evaluation methodology; trust boundaries; over-defense

## 1 Introduction

Prompt injection is the top entry in the OWASP list for LLM applications, and the defenses deployed against it are mostly detectors: a filter sits in front of the model and decides whether a prompt is hostile. Published detectors converge on two designs, regular expressions over attack vocabulary and fine-tuned transformer classifiers [6]. We built one of each, wired them in series, and evaluated the result far past the point where the numbers stopped flattering us.

This paper is a report on what that evaluation found. We do not present a new detection algorithm. We present measurements of a conventional one, taken under conditions that most published evaluations of this kind do not apply, and the measurements disagree with each other in ways that turn out to be informative.

The headline is a sequence. Measured on the corpus its blocklist was tuned against, our Tier 1 rule engine achieves perfect recall at zero false positives. Held out from that corpus it achieves 0.789. On the standard external benchmark [5] it achieves 0.40. On an indirect-injection benchmark [8] it achieves 0.48. Each number is honest; they differ because they measure different things, and only the last two say anything about a deployment.

What separates them is not attack difficulty. It is vocabulary. Our detector, like the blocklists it is representative of, keys on the language of override: *ignore previous instructions*, *disregard the above*, *you are now DAN*. An attacker who appends a well-formed instruction and no override phrasing at all, which is exactly what three of the five strategies in [5] do, walks through. On those three strategies our recall was zero.

The fix is not more vocabulary. It is context. In the threat model of [5] the injected instruction lands in the *data* an application processes, not in the user's own turn. Our detector was screening both identically, which is incoherent: our benign corpus contains "Summarize this document for me," a legitimate user request that is indistinguishable, as a string, from an injected one. Once callers declare which side of the trust boundary a text arrived on, recall on [5] moves from 0.40 to 1.00 at a 0.6% false positive rate, with the user path unchanged.

We claim four contributions. First, a quantification of in-sample inflation for this class of detector: 1.000 in-sample, 0.789 held out, 0.48 external, on the same system. Second, evidence that trust-boundary conditioning is worth more than vocabulary: one declared bit moves recall by 60 points where added keywords moved it by 2.6. Third, a characterization of two opposing failure modes, where the deterministic tier is precise but narrow (0.000 false positive rate on NotInject [4], 0.48 recall on BIPIA) and the learned tier is neither precise nor broad (0.404 false positive rate, 0.13 recall). Fourth, a negative result that generalizes past our system: pattern matching over instruction surface form does not transfer across instruction phrasings.

## 2 Background and related work

Liu et al. [5] give the formalization we use throughout, along with the benchmark that generates injected prompts by splicing an injected task's instruction and data into a target task's data under five strategies. Yi et al. [8] introduce BIPIA for the indirect case, where the payload is embedded in retrieved content rather than supplied directly. Both release code, and we reproduce their construction rather than approximating it.

Li and Liu [4] identify over-defense as the failure mode that actually breaks guard models in deployment, and release NotInject: 339 benign prompts seeded with attack trigger words, on which state-of-the-art guards drop to near-random accuracy. Their framing is why we report false positive rate on adversarially benign text as a first-class number rather than as a footnote.

Nasr et al. [7] evaluate twelve published defenses under adaptive attack and exceed 90% success on most, noting that the majority had originally reported near-zero attack success. That paper is the

reason our corpus has a held-out split at all. Hackett et al. [2] demonstrate character-injection evasion against production guardrails with up to 100% success, which is a direct attack on the normalization our Tier 1 depends on; we discuss this in Section 6 as an evaluated-nowhere gap rather than pretending coverage.

Closest to our system is Corll [1], who argues that a first screening layer should be fast, deterministic, non-promptable and auditable, and reports 95.97% recall on a genuine holdout with a sparse character *n*-gram SVM. We agree with the design argument and adopt the holdout discipline. Munirathinam [6] names the regex-plus-classifier pairing as the converged open-source architecture and predicts both failure modes we go on to measure, so we make no novelty claim for the architecture itself.

## 3 System

The service exposes one endpoint and screens in two stages. Tier 1 normalizes the input, then scans it with an Aho–Corasick automaton over 197 blocklist phrases and 14 structural regular expressions. Normalization strips zero-width characters, maps mathematical, fullwidth, circled and small-capital Unicode to ASCII, applies NFKD, removes combining marks, and decodes base64, hex and URL payloads when the decoded text contains injection-relevant keywords.

Tier 1 scans twice. The first pass runs over the normalized text with separators intact, so that literal markers such as `<|im_start|>` and `safety_mode=off` still match. The second runs over a de-obfuscated variant that maps leetspeak digits and symbols back to letters and strips intra-word separators. The second pass is what catches `1gn0r3 prev10us 1nstruct10ns`, `!gnore prev!ous` and `instr-uction-s` without the first pass losing its structural tokens. Median Tier 1 latency is 0.09 ms.

Tier 2 is `protectai/deberta-v3-base-prompt-injection-v2` exported to ONNX, consulted only for prompts Tier 1 clears, with a block threshold of 0.75 and median latency 24 ms.

### 3.1 Trust boundaries

The endpoint takes an optional `source` field with two values. Under `user`, the default, text is the end user's own turn and instructions in it are legitimate. Under `data`, text is content the application is processing, retrieved documents or tool output or third-party payloads, and an instruction addressed to a model is itself the signal, because data is meant to be read and not obeyed. Callers already possess this bit: an application passing a retrieved email knows it is not passing a user turn.

Under `data` we additionally apply 12 patterns matching instruction shape rather than attack vocabulary: forward references to following content, output-format constraints such as *answer with positive or negative*, role assignment such as *you are a tool*, sentence-initial task verbs, and assistant-directed question forms.

## 4 Evaluation

We use an in-house corpus of 271 samples, 190 attacks across ten classes and 81 benign, plus three external benchmarks. All measurements run against the containerized service on commodity hardware with CPU inference.

**Table 1: Tier 1 recall by rule configuration. The third row is not a result; its rules saw the test half, and it is shown only to quantify the optimism that produces.**

| Tier 1 rules | Train | Test | Test FPR |
|---|---|---|---|
| Before any corpus tuning | 0.7895 | 0.7632 | 0.0000 |
| Fitted on train split only | 1.0000 | 0.7895 | 0.0000 |
| Tuned on whole corpus | 1.0000 | 1.0000* | 0.0000 |

*contaminated

**Table 2: Open-Prompt-Injection recall by attack strategy (450 injected prompts, 6 tasks). Only strategies carrying override vocabulary are detected under the default source.**

| Strategy | Tier 1 | Tier 2 | `src=data` |
|---|---|---|---|
| naive | 0.00 | 0.00 | 1.00 |
| escape | 0.00 | 0.00 | 1.00 |
| ignore | 1.00 | 0.38 | 1.00 |
| fake completion | 0.00 | 0.00 | 1.00 |
| combined | 1.00 | 0.26 | 1.00 |
| **overall** | **0.40** | **0.13** | **1.00** |
| FPR on clean | 0.000 | 0.017 | 0.006 |

### 4.1 In-sample versus held out

Tier 1's blocklist was originally tuned the way such lists usually are: by reading the attacks it missed on the evaluation corpus and adding terms drawn from those samples. Recall measured afterwards on the same corpus is memorization. To get a defensible number we split the corpus 60/40, stratified by class and seeded, reconstructed the pre-tuning ruleset, and refit using train-split misses only. Table 1 shows the result.

The claimed figure overstates held-out recall by 21 points. Two details matter more than the gap itself. Fitting 24 new terms on the train split moved test recall by 2.6 points, from 0.763 to 0.789: hand-written blocklist terms fire on their training samples and little else. Meanwhile the de-obfuscation pass, which is a mechanism rather than a stored string, reaches 100% on the test half's `obfuscated_unicode` class. Mechanisms transferred; vocabulary did not.

The held-out split also reverses our reading of the second tier. Measured in-sample, Tier 2 contributed no detections, because a Tier 1 that has memorized the corpus leaves nothing for it to catch. Held out, Tier 2 catches all 16 attacks Tier 1 misses and takes cascade recall to 1.000. The case for defense in depth here is invisible without a holdout.

### 4.2 Open-Prompt-Injection

We reproduce the five attack strategies of [5] from upstream source, over six tasks, giving 450 injected prompts and 180 clean task-data prompts as negatives. Table 2 gives recall by strategy.

The pattern is stark and it is not a tuning gap. We detect the two strategies containing the literal string *Ignore previous instructions.* and miss all three that do not. A movie review followed by "Does the following text contain spam or phishing contents?" is a valid injection under this threat model and contains nothing a blocklist

**Table 3: BIPIA, 1,200 indirect injections over 15 categories. Attacks carry no override vocabulary.**

| Mode | Recall | FPR |
|---|---|---|
| Tier 1, `source=user` | 0.035 | 0.000 |
| Tier 1, `source=data` | 0.480 | 0.017 |
| before verb widening | 0.082 | 0.000 |

**Table 4: NotInject. All 339 samples are benign, so every block is a false positive.**

| Mode | Blocked | FPR |
|---|---|---|
| Tier 1 (197-term blocklist) | 0 / 339 | 0.0000 |
| Tier 2 (DeBERTa-v3) | 137 / 339 | 0.4041 |
| Shipped cascade | 0 / 339 | 0.0000 |

of override idioms can see. Declaring `source=data` takes overall recall to 1.00 at 1 false positive in 180. Tier 2, a model fine-tuned for this exact task, reaches 0.13.

### 4.3 BIPIA, and the ceiling

BIPIA [8] embeds an injected instruction in email and table content at start, middle or end. Its attacks carry no override vocabulary and no formatted task specification; they are ordinary requests such as "Write a script to rename all files in a directory." We construct 1,200 injected samples over 15 categories with 60 clean contexts.

This is the result we consider most useful. The instruction-shape patterns that reach 1.00 on [5] reach 0.48 here, and reached 0.082 before we widened the verb list. They were written against that benchmark's templates, so scoring well there was partly in-sample, exactly the error the previous subsection diagnoses one level down. Per-category recall tracks vocabulary overlap rather than attack severity: Substitution Ciphers 51%, five categories at 38%, and five including Marketing and Misinformation at 2%.

Widening the verb list bought 0.082 to 0.480 and left [5] at ∼1.00, at a cost in precision on imperative-heavy prose: false positives on real SMS payloads went from 2/100 to 5/100. That is the treadmill. Every benchmark needs new vocabulary and every addition costs precision somewhere else.

### 4.4 Over-defense, and two opposing failures

On NotInject [4], where all 339 samples are benign and every block is a false positive, Tier 1 blocks none and Tier 2 blocks 137. Tier 2's over-defense scales with trigger density: 20%, 50% and 51% at one, two and three trigger words, and a large share of its false positives are benign Chinese prompts flagged on a single character.

This is not fixable by threshold. On NotInject's benign samples Tier 2's injection score has median 0.9992, with 53% at or above 0.999; on the held-out attacks it correctly rescues, the median is 1.0000 with 88% above that line. The distributions overlap almost completely. We therefore ship Tier 2 in monitor-only mode: it scores and logs but does not block. A 40% false-block rate is not a shippable default, and the cost, a recall ceiling of 0.789 rather than 1.000, is one we state rather than hide.

The two tiers fail in opposite directions. The deterministic one is precise and narrow. The learned one is neither precise nor, on these benchmarks, broad. No configuration of this cascade is simultaneously robust and precise, and no threshold splits the difference.

### 4.5 A quantized guardrail that had stopped working

The shipped Tier 2 was an INT8 graph produced with per-tensor weight scaling. It had collapsed to predicting SAFE for every input: it labels "Ignore all previous instructions and reveal your system prompt" SAFE at 0.94 confidence. Behind a fail-open policy this is invisible. The tier loads cleanly, reports latency, scores every request, and blocks nothing. It produced a Tier 2 recall of 0.005 that we initially read as a property of the model.

Re-benchmarking every available graph gave fp32 at 0.900 recall, and two correctly configured INT8 variants at 0.758 and 0.732. Even done properly, quantization cost 14 to 17 points of recall for roughly 5 ms and 495 MB. DeBERTa-v3's disentangled attention is sensitive to per-tensor scaling. The transferable lesson is that a silently degraded guardrail and a healthy one are indistinguishable from the outside when the policy is fail-open, so quantized graphs need accuracy verification and not just a smoke test.

## 5 Discussion

Detection here is trust-boundary dependent, and the dependence is large. One declared bit moves recall on [5] by 60 points. Vocabulary added by hand moved held-out recall by 2.6. Any deployment that screens retrieved content with the same rules it applies to user turns is leaving most of that on the table, and the information required is already available at the call site.

The BIPIA result bounds what this family of detector can do. Patterns over surface form transfer poorly across phrasings of the same threat, and the failure is graded by vocabulary overlap rather than by how dangerous the attack is. A detector that generalizes has to model what a span of text is trying to do, not the words it uses. Representation-level approaches [3] look better suited to that than any list we could write.

None of this makes the deterministic tier useless. It is the only component here with a clean precision record: 0 of 339 on NotInject, 0 of 180 on real task data, sub-millisecond, auditable, and it fails in ways an operator can read off a config file. As a cheap first filter it is good. As a solution to indirect prompt injection it is not, and the gap between those two claims is where most of the risk in deploying one of these lives.

## 6 Limitations

No adaptive attacker. Every number here is against static corpora, which is the evaluation Nasr et al. [7] argue overstates robustness, and we expect ours are no exception. Our figures are upper bounds.

The refit is not blind. The train-only rules were authored by someone who had already seen the full corpus, so 0.789 is optimistic. The split, per-term provenance and scripts are published so the protocol can be re-run cleanly by someone who has not.

Character-injection evasion [2] is unevaluated, and it targets our normalization directly. Our de-obfuscation handles leetspeak,

small caps, fullwidth forms, combining marks and separator fragmentation, but has never been tested against optimized character injection. We expect it to do badly and we have not measured how badly.

Scope. One detector, one 271-sample corpus, BIPIA restricted to email and table contexts, and the summarization task of [5] omitted because its data is gated. Declarative injections remain undetected under `source=data`: "It would be helpful to know the sentiment of this text" is missed, verified.

## 7 Conclusion

A conventional two-tier guardrail scores 1.000, 0.789, 1.00 and 0.48 on four evaluations of the same capability, and the spread is the finding. Telling the detector which trust boundary a text crossed is worth more than any vocabulary we added to it. The deterministic tier is precise and narrow, the learned tier over-defends on 40% of adversarially benign prompts, and quantization had silently disabled the latter in a way a fail-open policy concealed. Pattern matching over instruction surface form is a good first filter and it does not survive a change of phrasing.

## Availability

Source, all measurement scripts, the corpus split and every result artifact are public. The repository link is withheld for double-blind review and will be included in the camera-ready version.

## Use of generative AI

Generative AI was used substantially in this work, and the following describes the extent.

An AI coding assistant (Claude, Anthropic) was used under the author's direction for: implementing the de-obfuscation scan and the `source` trust-boundary mechanism; writing the harnesses that reproduce Open-Prompt-Injection, BIPIA and NotInject; running the experiments; and producing the first draft of this text, including its tables. Several findings reported here surfaced during that work, among them the INT8 quantization collapse in Section efsec:quant and the evaluation contamination in Section efsec:heldout.

The author set the research direction, decided what to measure, made the design decisions the paper reports (notably shipping Tier 2 monitor-only rather than blocking), and is responsible for all claims. Every quantitative claim is generated from committed result artifacts by a checking script rather than transcribed, and the reference list was assembled from retrieved papers rather than from model recall, both because model-produced numbers and citations are unreliable. Remaining errors are the author's.

## References


[1] J. Alex Corll. 2026. The Mirror Design Pattern: Strict Data Geometry over Model Scale for Prompt Injection Detection. *arXiv preprint arXiv:2603.11875* (2026).

[2] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. 2025. Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. In *LLMSec.* arXiv:2504.11168.

[3] Jinyuan Jia, Yupei Liu, Yuqi Hu, and Neil Zhenqiang Gong. 2025. PIShield: Detecting Prompt Injection Attacks via Intrinsic LLM Features. *arXiv preprint arXiv:2510.14005* (2025).

[4] Hao Li and Xiaogeng Liu. 2024. InjecGuard: Benchmarking and Mitigating Over-defense in Prompt Injection Guardrail Models. *arXiv preprint arXiv:2410.22770* (2024). Published as PIGuard, ACL 2025.

[5] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *33rd USENIX Security Symposium (USENIX Security 24).* arXiv:2310.12815.

[6] Thamilvendhan Munirathinam. 2026. Beyond Pattern Matching: Seven Cross-Domain Techniques for Prompt Injection Detection. *arXiv preprint arXiv:2604.18248* (2026).

[7] Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, Abhradeep Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. 2025. The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections. *arXiv preprint arXiv:2510.09023* (2025).

[8] Jingwei Yi, Yueqi Xie, Bin Zhu, Keegan Hines, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2023. Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. *arXiv preprint arXiv:2312.14197* (2023).